

# FOSS tools for RTL simulation and synthesis

## Icarus Verilog

### Introduction

Icarus Verilog, often referred to as iVerilog, is an open-source hardware description language (HDL) simulator used for the simulation and verification of digital designs. It supports the Verilog hardware description language, which is widely used for the design and verification of digital circuits. Icarus Verilog is used to simulate the behavior of digital circuits written in Verilog before they are synthesized and implemented on actual hardware.

### Building/Installing Icarus Verilog from Source

Although pre build packages exist for most linux distros we will focus on installing it from source as other FOSS EDA based tools are dependent upon iVerilog in various stages.

1. Create a tools folder in \$HOME directory.

```
[ -d ~/Tools ] || mkdir -p ~/Tools  
cd ~/Tools
```

2. Install all the pre requisites tools and packages required.

- a. For Linux

```
sudo apt install -y build-essential bison flex gperf termcap  
autoconf lib32readline7 lib32readline-dev
```

- b. For MacOS

```
brew install bison flex gcc gmake gperf autoconf
```

3. Clone the iVerilog repo

```
git clone https://github.com/steveicarus/iverilog.git  
cd iverilog
```

4. Generate the "configure" file.

```
sh autoconf.sh
```

5. Compile the source code.

```
./configure  
make
```

6. Run a simple test before installation.

```
make check
```

7. Install iVerilog

```
sudo make install
```

8. Clean any and all unnecessary files.

```
make clean  
make distclean
```

## Yosys

### Introduction

Yosys is an open-source framework for Verilog RTL (Register-Transfer Level) synthesis. It is commonly used for digital synthesis, formal verification, and other tasks related to digital design. Yosys takes as input a hardware description in Verilog (or a subset of SystemVerilog) and produces a synthesized netlist as output, which can then be used for implementation on an FPGA or other digital hardware.

### Building/Installing Icarus Verilog from Source

Although pre build packages exist for most linux distros we will focus on installing it from source as other FOSS EDA based tools are dependent upon Yosys in various stages.

1. Create a tools folder in \$HOME directory.  

```
[ -d ~/Tools ] || mkdir -p ~/Tools  
cd ~/Tools
```
2. Install all the pre requisites tools and packages required.
  - a. For Linux  

```
sudo apt install -y build-essential python3 bison flex gawk git tcl-dev graphviz xdot libreadline-dev libffi-dev pkg-config libboost-system-dev libboost-python-dev libboost-filesystem-dev zlib1g-dev
```
  - b. For MacOS  

```
brew install gcc gmake bison flex gawk libffi git graphviz pkg-config python3 tcl-tk xdot bash boost-python3
```
3. Clone the Yosys repo  

```
git clone https://github.com/YosysHQ/yosys.git  
cd yosys
```
4. Configure the build system to use a specific compiler.  

```
make config-gcc
```
5. Build yosys  

```
make
```
6. Install yosys  

```
sudo make install
```
7. Execute tests (iverilog required to be built from source)  

```
make test
```
8. Clean any and all unnecessary files.  

```
make clean clean-abc
```

### Simulating a design using Icarus Verilog

In this exercise we are going to design an N-bit 2x1 mux in System Verilog, write its testbench and then simulate it using Icarus Verilog.

RTL Design File (mux.sv)

Below added is the RTL code for an N-bit 2x1 mux.

```
`timescale 1ns/100ps

module mux2x1 #(parameter DATA_WIDTH=1) (
    input  logic [DATA_WIDTH-1:0] in_a,
    input  logic [DATA_WIDTH-1:0] in_b,
    input  logic                    sel,
    output logic [DATA_WIDTH-1:0] out
);

    assign out = (sel)? in_a : in_b;

endmodule
```

Testbench file (tb\_mux.sv)

Below attached is the testbench code for an N-bit 2x1 mux.

```
`timescale 1ns/100ps

module tb_mux2x1();

    localparam DATA_WIDTH = 8;
    logic [DATA_WIDTH-1:0] in_a;
    logic [DATA_WIDTH-1:0] in_b;
    logic                    sel;
    logic [DATA_WIDTH-1:0] out;

    mux2x1 #(.DATA_WIDTH(DATA_WIDTH)) dut_mux2x1(
        .in_a (in_a ),
        .in_b (in_b ),
        .sel  (sel  ),
        .out  (out  )
    );

    initial begin
        for (int i=0; i<10; i=i+1) begin
            in_a    = $urandom_range('h0,'hff);
            in_b    = $urandom_range('h0,'hff);
            sel     = i%2;
            #1;
            $display("| Time: %2d | Input_A: %h | Input_B: %h | Sel: %1d |",
Output: %h |", $time, in_a, in_b, sel, out);
            end
            $finish;
        end
    end
```

```

initial begin
    $dumpfile("01_mux/01_mux.vcd");
    $dumpvars(0,tb_mux2x1);
end

```

endmodule

### Running the testbench

To run the testbench using iverilog run the following commands.

```

iverilog -o <vvp_file_name>.vvp -Wall -Winfloop -gno-shared-loop-index -
g2012 <design_file_name>.sv <tb_top_file_name>.sv

```

```

vvp <vvp_file_name>.vvp

```

### Synthesizing the design

There are two ways to synthesize the design and generate the netlist using yosys. The first is to either open yosys in the terminal and run all commands sequentially. The second way is to create a yosys script <script\_name>.ys and run it from BASH shell. We will focus on the 2<sup>nd</sup> way.

Create a file <script\_name>.ys with the following text

```

read -sv <design_file_name>.sv
hierarchy -check -auto-top
flatten
proc; opt_expr; opt_clean
check; opt -nodffe -nosdff
fsm; opt
wreduce
peepopt; opt_clean
techmap
alumacc
share; opt
memory -nomap; opt_clean
opt -fast -full
memory_map; opt -full
techmap; opt -fast
abc -fast; opt -fast
clean
write_verilog <Verilog_netlist_filename>.v
json -aig -o <json_netlist_filename>.json
stat -tech cmos -width
show -format dot -viewer xdot -stretch -width -colors 10000 -signed -prefix
<project_folder_path>

```

Run the script

```

yosys <script_name>.ys

```